RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

www.rvce.edu.in
Tel: +91-80-68188100
+91-80-68188111
+91-80-68188112

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

MULTI-PHASE INTELLIGENT ALGORITHMIC THREAT URL DETECTION FRAMEWORK

Design and Analysis of Algorithms– CD343AI

Experiential Learning (Lab)

REPORT

Submitted by

Tanish Champawat **1RV24IS136**

Vivaan Hooda **1RV24IS152**

Under the guidance of

Swetha S

Affiliation

In partial fulfilment for the award of degree of

Bachelor of Engineering

in

Department of Information Science and Engineering

2025-2026

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project work titled ***'Multi-Phase Intelligent Algorithmic Threat URL Detection Framework'*** is carried out by **TANISH CHAMPAWAT(1RV24IS136), VIVAAN HOODA (1RV24IS152)** who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfilment for the award of degree of **Bachelor of Engineering in Information Science and Engineering** of the **Visvesvaraya Technological University**, Belagavi during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution for the said degree.

Signature of Guide
Swetha S

Signature of Head of the Department
Dr. G S Mamatha

External Viva

Name of Examiners	**Signature with Date**
1	
2	

RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

DECLARATION

We**, TANISH CHAMPAWAT(1RV24IS136),VIVAAN HOODA(1RV24IS152),** students of Fourth semester B.E., department of Information Science Engineering, RV College of Engineering, Bengaluru, hereby declare that Experiential Learning (Lab) titled ***'Multi-Phase Intelligent Algorithmic Threat URL Detection Framework'*** has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Information Science and Engineering** during the academic year 2025-26

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering®, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru

Date:

Name	**Signature**
1. TANISH CHAMPAWAT (1RV24IS136)	
2. VIVAAN HOODA (1RV24IS152)	

ABSTRACT

The rapid growth of internet services has significantly increased the risk of phishing attacks and malicious websites. Cybercriminals frequently use deceptive URLs to steal sensitive information such as usernames, passwords, and financial details. Traditional blacklist-based detection systems are often unable to identify newly generated malicious URLs. To address this challenge, various machine learning and algorithm-based techniques have been proposed for URL threat detection. This project proposes a **Multi-Phase Intelligent Algorithmic Threat URL Detection Framework** that combines Design and Analysis of Algorithms (DAA) concepts with Artificial Intelligence to accurately classify URLs as Safe, Suspicious, or Malicious. The primary objective is to provide efficient and real-time threat detection through a multi-layered analysis pipeline.

The proposed framework employs multiple algorithms to analyze different characteristics of a URL. URL Expansion is used to reveal the actual destination of shortened links, while Breadth First Search (BFS) and Depth First Search (DFS) are used to analyze redirect chains and URL relationships. Boyer-Moore Pattern Matching identifies phishing-related keywords such as login, verify, and secure. A Neural Network classifier extracts and evaluates multiple URL features including URL length, entropy, keyword score, HTTPS presence, and suspicious domain indicators to generate a threat probability. Greedy Optimization prioritizes high-risk URLs for analysis, Bloom Filter enables fast blacklist lookup, Dijkstra's Algorithm assists in redirect path analysis, and Heapsort is used to rank URLs according to their threat scores. These algorithms collectively improve detection efficiency and accuracy.

A prototype of the proposed system was developed using Python, Flask, TensorFlow, React, and related web technologies. The system provides an interactive user interface where users can submit URLs for analysis. The framework processes URLs through multiple phases and generates a final threat score along with detailed analysis results. Performance benchmarking and system analytics modules were also implemented to evaluate the efficiency of various algorithms. Experimental results demonstrate that the framework can effectively identify suspicious URLs while providing real-time analysis and visualization of threat indicators.

The developed framework successfully demonstrates the application of DAA concepts and Machine Learning in the field of cybersecurity. By integrating graph traversal, pattern matching, optimization techniques, probabilistic data structures, sorting algorithms, and neural networks, the system provides a comprehensive solution for malicious URL detection. Future enhancements may include the integration of real-time threat intelligence feeds, larger phishing datasets, advanced deep learning models, automated blacklist updates, and browser extensions for real-time protection. These improvements can further increase detection accuracy, scalability, and practical deployment capabilities.

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1.1 Background

The Internet has become an essential part of modern life, enabling communication, online banking, e-commerce, education, and various digital services. Along with the rapid growth of online activities, cyber threats have also increased significantly. One of the most common cyber threats is phishing, where attackers create fake websites and malicious URLs to deceive users into revealing sensitive information such as usernames, passwords, banking credentials, and personal data.

A malicious URL is a web address designed to direct users to harmful websites. These URLs often imitate legitimate websites and use deceptive techniques such as URL shortening, domain spoofing, suspicious keywords, and multiple redirections. Traditional URL detection systems rely heavily on static blacklists that contain previously identified malicious websites. However, cybercriminals continuously generate new malicious URLs, making blacklist-based approaches insufficient for real-time protection.

To address these limitations, intelligent URL analysis systems have been developed using machine learning and advanced algorithms. Such systems analyze various characteristics of URLs and identify suspicious patterns that may indicate malicious behavior. The integration of Artificial Intelligence and Design and Analysis of Algorithms (DAA) provides an efficient solution for modern cybersecurity challenges.

1.2 Problem Statement

Traditional URL detection mechanisms face several challenges:

1. Inability to detect newly generated phishing URLs.
2. High dependency on manually maintained blacklists.
3. Difficulty in analyzing complex redirect chains.
4. Inefficient processing of large numbers of URLs.
5. Limited capability to identify suspicious URL structures.

As cyber threats continue to evolve, there is a need for an intelligent framework capable of analyzing URLs using multiple algorithmic approaches and machine learning techniques.

1.3 Proposed Solution

The proposed project, “Multi-Phase Intelligent Algorithmic Threat URL Detection Framework”, introduces a multi-layered approach for malicious URL detection. The framework combines several DAA algorithms with a Neural Network-based machine learning model to perform comprehensive URL analys

The system operates through multiple phases including URL Expansion, Graph Traversal, Pattern Matching, Neural Network Classification, Greedy Optimization, Bloom Filter Verification, and Heapsort Ranking.

Each phase contributes unique information that helps determine whether a URL is safe, suspicious, or malicious. Unlike traditional systems that rely solely on blacklists, the proposed framework analyzes URL characteristics such as length, entropy, keyword patterns, redirect behavior, domain properties, and threat scores. This allows the system to identify both known and previously unseen malicious URLs.

1.4 Significance of the Project

The significance of this project lies in its ability to combine classical algorithms and modern artificial intelligence techniques within a single cybersecurity framework.

The project demonstrates practical applications of:

- Breadth First Search (BFS)
- Depth First Search (DFS)
- Boyer-Moore Pattern Matching
- Dijkstra's Algorithm
- Greedy Algorithms
- Bloom Filters
- Heapsort
- Neural Networks

By integrating these techniques, the system achieves efficient URL analysis, rapid threat detection, and improved decision-making capabilities.

1.5 Objectives

The primary objectives of the project are:

1. To detect malicious and phishing URLs in real time.
2. To analyze URL redirect chains using BFS and DFS algorithms.
3. To identify suspicious keywords through pattern matching techniques.
4. To classify URLs using a Neural Network-based machine learning model.
5. To implement fast blacklist verification using Bloom Filters.
6. To prioritize suspicious URLs using Greedy Optimization.
7. To rank detected threats using Heapsort.

8. To provide a user-friendly web interface for URL analysis

9. To demonstrate the practical implementation of DAA concepts in cybersecurity

1.6 Scope of the Project

The proposed framework focuses on URL-based threat detection and analysis. The system accepts URLs as input and evaluates them through multiple algorithmic stages before generating a final threat assessment.

The project can be applied in:

- Cybersecurity systems
- Web browsers
- Corporate networks
- Email security solutions
- Educational research
- Threat intelligence platforms

The framework can also serve as a foundation for future research involving advanced machine learning models and real-time threat intelligence integration.

1.7 Chapter Summary

This chapter introduced the problem of malicious URL detection and discussed the limitations of traditional approaches. The proposed Multi-Phase Intelligent Algorithmic Threat URL Detection Framework was presented as an intelligent solution that combines multiple DAA algorithms and machine learning techniques to improve threat detection accuracy and efficien

CHAPTER 2

SOLUTION DESIGN

2.1 Overview of the Proposed System

The proposed system, **Multi-Phase Intelligent Algorithmic Threat URL Detection Framework**, is designed to detect malicious and phishing URLs through a combination of Design and Analysis of Algorithms (DAA) concepts and Machine Learning techniques. Instead of relying solely on traditional blacklist-based methods, the framework performs a comprehensive analysis of URL characteristics, redirect behavior, keyword patterns, and threat indicators before generating a final verdict.

The system follows a multi-phase architecture where each module performs a specific task. The output of one phase serves as input to the next phase, enabling progressive threat analysis. The final result is presented to the user through an interactive web-based dashboard.

2.2 System Architecture

The overall architecture of the proposed framework consists of the following phases:

1. User Input Module
2. URL Expansion Module
3. Graph Traversal Module (BFS & DFS)
4. Pattern Matching Module (Boyer-Moore)
5. Neural Network Classification Module
6. Greedy Optimization Module
7. Bloom Filter Verification Module
8. Heapsort Ranking Module
9. Final Threat Assessment Module
10. Analytics and Visualization Dashboard

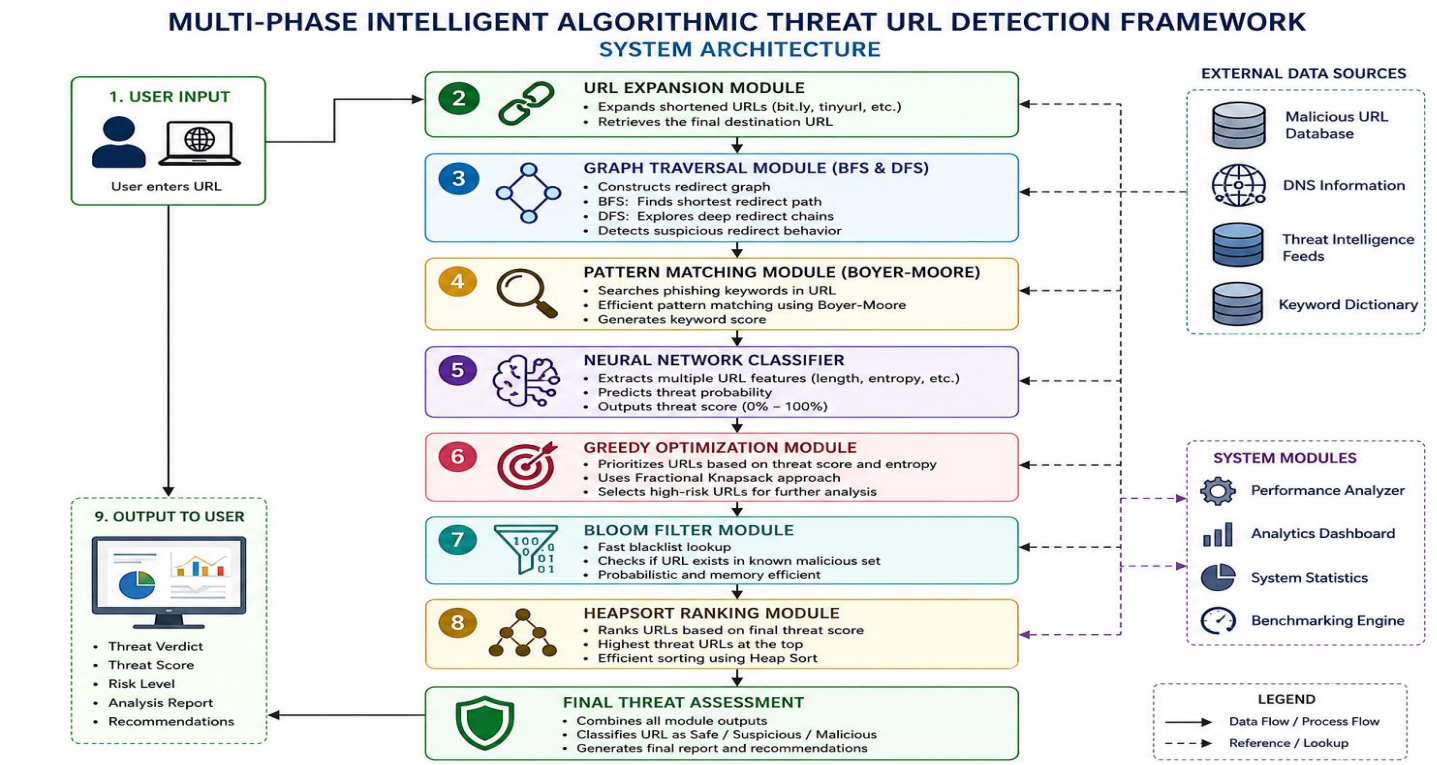

Figure 2.1: *Architecture of Multi-Phase Intelligent Algorithmic Threat URL Detection Framework*

The architecture follows a pipeline-based approach where the URL passes through multiple stages of analysis before a final threat score is generated.

2.3 User Input Module

The User Input Module acts as the entry point of the system. Users enter a URL through the web interface developed using React. The system validates the URL format and ensures that the input is suitable for analysis.

The module performs the following functions:

- Accepts URL input from users.
- Performs basic validation.
- Sends URL to the processing pipeline.
- Displays final analysis results.

This module provides a simple and user-friendly interface for threat analysis.

2.4 URL Expansion Module

Cybercriminals often use URL shortening services such as Bit.ly, TinyURL, and T.co to hide malicious destinations. The URL Expansion Module resolves shortened URLs and reveals the actual destination before further analysis.

Functions performed:

- Expands shortened URLs.

- Detects multiple redirections.
- Retrieves final destination URLs.
- Prevents URL obfuscation attacks.

This phase ensures that hidden malicious destinations cannot bypass the security analysis process.

2.5 Graph Traversal Module (BFS and DFS)

After URL expansion, the framework analyzes URL redirect chains using graph traversal algorithms.

Breadth First Search (BFS)

BFS explores redirect relationships level by level. It helps identify:

- Redirect structures.
- Connected URL nodes.
- Redirect depth.
- Reachability analysis.

Time Complexity:

$O(V + E)$

where:

- V = Number of URL nodes.
- E = Number of redirect relationships.

Depth First Search (DFS)

DFS follows redirect paths deeply before backtracking. It helps determine:

- Final destination URLs.
- Deep redirect chains.
- Suspicious redirect behavior.

Time Complexity:

$O(V + E)$

The combination of BFS and DFS allows comprehensive redirect analysis.

2.6 Pattern Matching Module

Phishing URLs frequently contain suspicious keywords such as:

- login
- verify
- secure
- bank
- update

- password

The system uses the Boyer-Moore Pattern Matching Algorithm to efficiently search for these keywords.

Advantages of Boyer-Moore:

- Faster than naive string matching.
- Skips unnecessary comparisons.
- Efficient for long strings.

The module generates a keyword threat score based on detected phishing-related terms.

2.7 Neural Network Classification Module

The Neural Network Module serves as the primary threat prediction component of the framework.

The system extracts approximately 25 features from each URL, including:

URL Structure Features

- URL Length
- Domain Length
- Subdomain Depth
- Dot Count
- Hyphen Count

Entropy-Based Features

- URL Entropy
- Domain Entropy
- Digit Ratio
- Special Character Ratio

Security Features

- HTTPS Presence
- Suspicious TLD
- Homograph Detection
- IP Address Usage

Threat Indicators

- Keyword Score
- Redirect Depth
- Chain Length
- Domain Age
-

These features are converted into numerical values and passed to a Neural Network developed using TensorFlow and Keras.

The Neural Network predicts a threat probability ranging from 0 to 100 percent.

Based on the probability score, URLs are classified as:

- Safe
- Suspicious
- Malicious

The Neural Network acts as the intelligent decision-making component of the framework.

2.8 Greedy Optimization Module

The Greedy Optimization Module prioritizes URLs based on threat level.

The module uses a Fractional Knapsack-inspired Greedy approach where URLs with higher threat scores receive higher priority.

Functions:

- Calculates priority scores.
- Identifies high-risk URLs.
- Optimizes resource utilization.
- Supports batch URL analysis.

In large-scale environments where thousands of URLs must be processed, Greedy Optimization ensures that the most dangerous URLs are analyzed first.

2.9 Bloom Filter Verification Module

Bloom Filter is a probabilistic data structure used for fast membership testing.

The module stores hashed signatures of known malicious URLs and performs rapid blacklist verification.

Advantages:

- Very low memory usage.
- Fast lookup operations.
- Scalable for large datasets.

Time Complexity:

O(1)

Bloom Filter helps identify URLs that are already known to be malicious.

2.10 Heapsort Ranking Module

After threat scores are generated, the Heapsort Module ranks URLs according to their threat levels.

Functions:

- Sorts URLs by threat score.
- Prioritizes dangerous URLs.
- Generates ranked threat reports.

Time Complexity:

O(n log n)

Although the demonstration analyzes a single URL at a time, the ranking module supports batch processing scenarios.

2.11 Final Threat Assessment Module

The Final Threat Assessment Module combines outputs from all previous phases.

Inputs:

- Redirect Analysis
- Keyword Score
- Neural Threat Probability
- Bloom Filter Results
- Ranking Information

Outputs:

- Safe
- Suspicious
- Malicious

The final threat score is displayed along with recommendations and security insights.

2.12 Dashboard and Visualization Layer

The frontend interface provides multiple dashboards:

URL Analyzer Dashboard

Displays:

- Threat Score
- Threat Signals
- Pipeline Results
- Neural Feature Analysis

Analytics Dashboard

Displays:

- Model Statistics
- Accuracy Metrics
- Dataset Information

Performance Dashboard

Displays:

- Algorithm Execution Times
- Throughput
- Latency
- Benchmark Results

System Statistics Dashboard

Displays:

- Algorithms Used
- Complexity Information
- Pipeline Components

The dashboard provides real-time visualization of the complete threat analysis process.

2.13 Algorithm Selection and Justification

Algorithm	Purpose	Justification
BFS	Redirect Analysis	Efficient graph traversal
DFS	Deep Redirect Analysis	Complete path exploration
Boyer-Moore	Keyword Detection	Fast pattern matching
Neural Network	Threat Classification	Learns complex URL patterns
Greedy Algorithm	URL Prioritization	Efficient resource allocation
Dijkstra	Redirect Path Analysis	Shortest suspicious path analysis
Bloom Filter	Blacklist Verification	Fast memory-efficient lookup
Heapsort	Threat Ranking	Efficient sorting of threat scores

2.14 Chapter Summary

This chapter presented the complete design of the proposed Multi-Phase Intelligent Algorithmic Threat URL Detection Framework. The architecture integrates graph traversal algorithms, pattern matching techniques, machine learning models, optimization strategies, probabilistic data structures, and sorting algorithms to provide comprehensive malicious URL detection. The combination of these modules ensures accurate, efficient, and scalable threat analysis

CHAPTER 3
IMPLEMENTATION DETAILS

3.1 Introduction

The implementation phase focuses on transforming the proposed design into a fully functional software system capable of detecting malicious and phishing URLs. The developed framework integrates multiple Design and Analysis of Algorithms (DAA) concepts with Machine Learning techniques to provide accurate and efficient URL threat detection. The system follows a modular architecture in which each analysis phase is implemented independently and integrated into a unified pipeline.

The implementation consists of two major components: the backend processing system and the frontend user interface. The backend is responsible for URL processing, feature extraction, algorithm execution, machine learning prediction, blacklist verification, and threat ranking. The frontend provides an interactive dashboard through which users can submit URLs and view analysis results.

3.2 Development Environment and Technology Stack

The framework was developed using modern web and machine learning technologies. Python was selected as the primary backend programming language because of its extensive support for machine learning and data processing. Flask was used to develop REST APIs and manage communication between the frontend and backend components.

The machine learning component was implemented using TensorFlow and Keras. These libraries provide efficient tools for developing and deploying neural network models. Data processing and numerical computations were supported using NumPy and Pandas.

The frontend was developed using React.js, JavaScript, HTML, CSS, Tailwind CSS, and Vite. React was selected because of its component-based architecture, which improves code reusability and maintainability. Tailwind CSS was used to create a responsive and visually appealing user interface.

The development process was carried out using Visual Studio Code, while package management and dependency installation were performed using npm and pip.

3.3 Backend Implementation

The backend forms the core processing unit of the framework. It is responsible for receiving URL requests, coordinating different analysis phases, generating threat scores, and returning results to the frontend.

The backend follows a modular architecture where each analysis phase is implemented as a separate module. This approach improves maintainability, scalability, and code readability.

The URL Expansion Module resolves shortened URLs and retrieves the actual destination URL. This helps prevent attackers from hiding malicious websites behind URL shortening services.

The Graph Traversal Module implements Breadth First Search (BFS) and Depth First Search (DFS) algorithms. These algorithms analyze redirect chains and URL relationships. BFS explores redirect structures level by level, while DFS follows redirect paths deeply to identify the final destination URL and redirect depth.

The Pattern Matching Module uses the Boyer-Moore algorithm to search for suspicious keywords such as login, verify, secure, bank, update, and password. The presence of these keywords contributes to the overall threat score.

The Neural Network Classification Module extracts multiple features from each URL and predicts the probability of the URL being malicious. This module acts as the primary decision-making component of the framework.

The Greedy Optimization Module prioritizes URLs according to their threat scores. In large-scale environments where numerous URLs are analyzed simultaneously, this module helps focus resources on high-risk URLs.

The Bloom Filter Module performs rapid blacklist verification. It stores hashed signatures of known malicious URLs and allows fast membership testing with minimal memory usage.

The Heapsort Ranking Module ranks URLs according to their threat scores. This helps security analysts identify the most dangerous URLs first when processing multiple URLs simultaneously.

3.4 Frontend Implementation

The frontend was developed to provide an intuitive and user-friendly interface for URL analysis. React.js was used to create a dynamic single-page application that communicates with the backend through REST APIs.

The homepage of the application serves as the URL Analyzer Dashboard. Users can enter a URL and initiate the analysis process. Once the analysis is completed, the system displays the threat score, threat classification, threat indicators, and results generated by each analysis phase.

The Analytics Dashboard presents machine learning information including dataset statistics, model performance metrics, and feature extraction details. This dashboard helps users understand how the Neural Network performs URL classification.

The Performance Dashboard provides benchmarking information related to the algorithms implemented within the framework. It displays execution time, latency, throughput, and performance statistics for different modules.

The System Statistics Dashboard summarizes the algorithms used in the framework and provides information regarding their purpose and computational complexity.

The frontend communicates with the backend using HTTP requests and dynamically updates the interface whenever new results are generated.

3.5 Machine Learning Implementation

The machine learning component is implemented using a Neural Network developed with TensorFlow and Keras. The primary objective of the model is to classify URLs as Safe, Suspicious, or Malicious based on extracted URL features.

Before classification, each URL undergoes a feature extraction process. Multiple characteristics of the URL are analyzed and converted into numerical values. Examples of extracted features include URL length, domain length, entropy, number of digits, HTTPS status, suspicious top-level domains, keyword score, and redirect depth.

These numerical features are supplied to the Neural Network model. During training, the model learns patterns associated with malicious and legitimate URLs. Once trained, the model can predict the probability that a new URL is malicious.

The output generated by the Neural Network is combined with results from other analysis modules to produce the final threat assessment.

3.6 API Integration

Communication between the frontend and backend is achieved through REST APIs. These APIs allow different components of the framework to exchange information efficiently.

The URL Analysis API receives a URL from the frontend, executes the complete analysis pipeline, and returns detailed threat information. The Analytics API provides machine learning statistics and system information, while the Performance API generates benchmark results and algorithm execution metrics.

The API-based architecture improves scalability and allows future integration with browser extensions, mobile applications, or external cybersecurity systems.

3.7 User Interface Screenshots

Figure 3.1: URL Analyzer Dashboard

Description: The URL Analyzer Dashboard serves as the primary interface of the framework. Users can submit URLs for analysis and view detailed threat assessment results including threat score, threat indicators, and pipeline outputs.

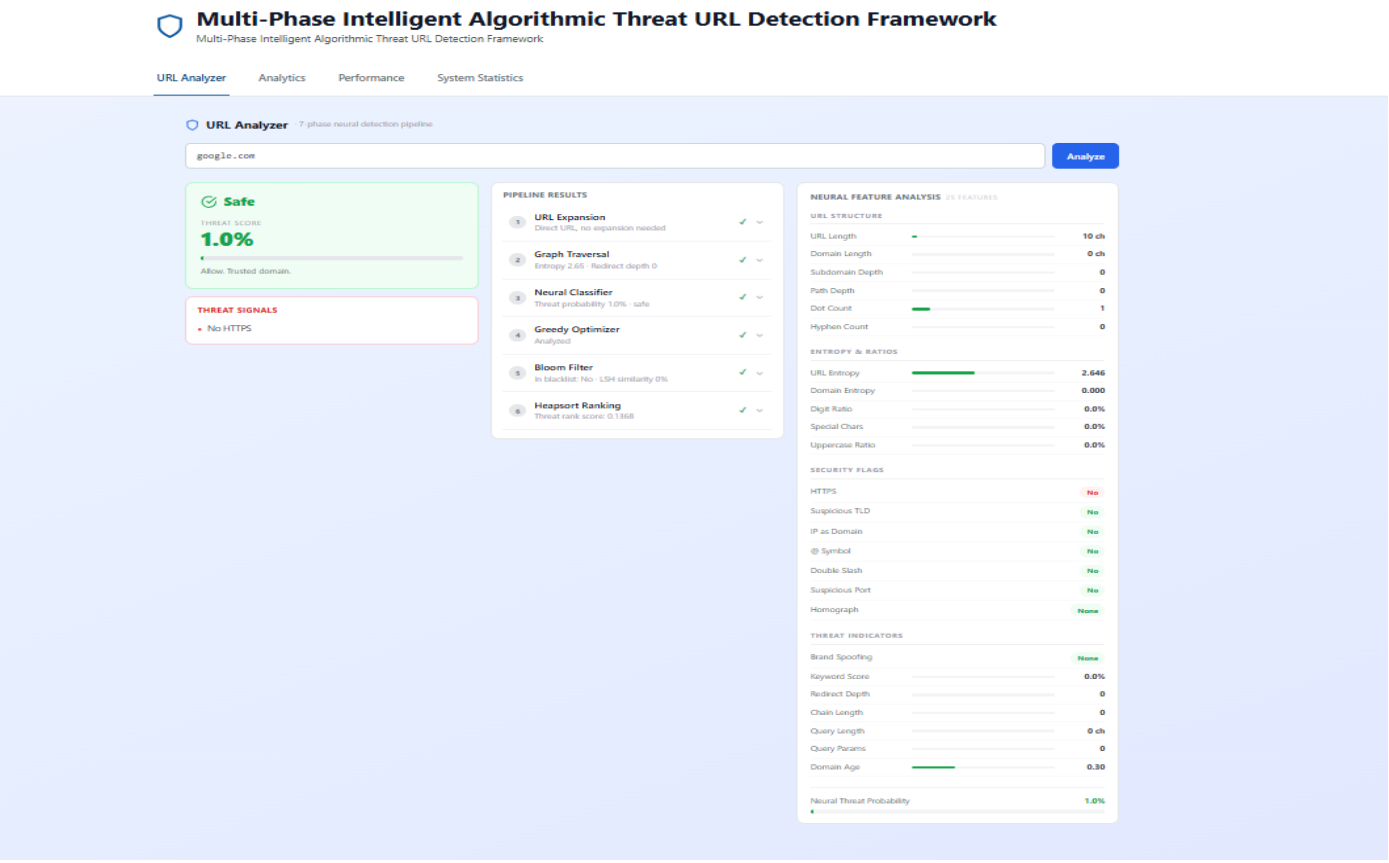

Figure 3.2: Threat Analysis Result

Description: This screen displays the final threat classification generated by the framework. It includes the calculated

threat score, identified threat signals, and detailed analysis performed by different modules.

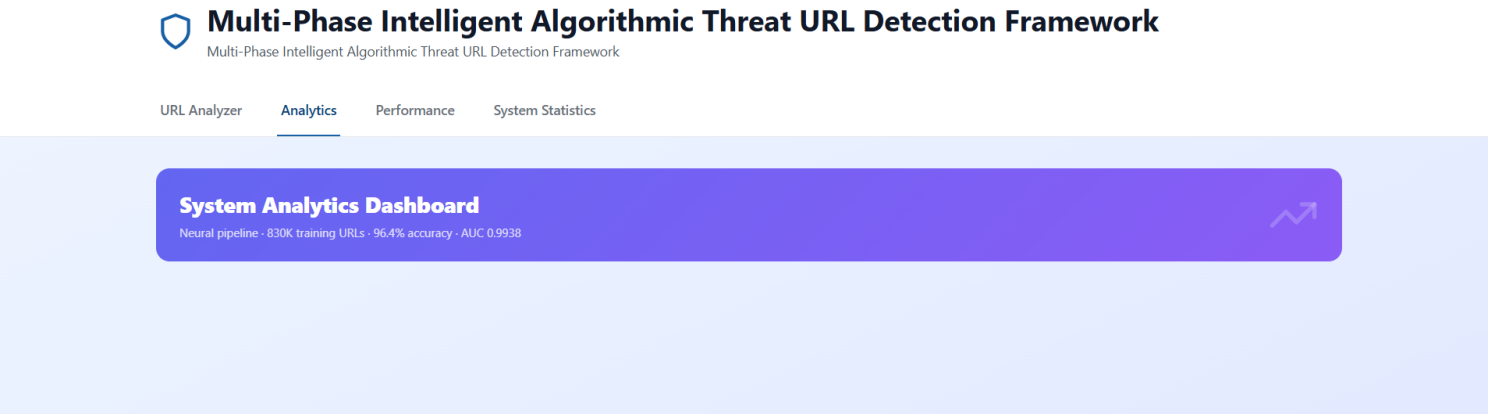

Figure 3.3: Analytics Dashboard

Description: The Analytics Dashboard presents machine learning statistics such as model accuracy, dataset information, feature extraction details, and classification performance metrics.

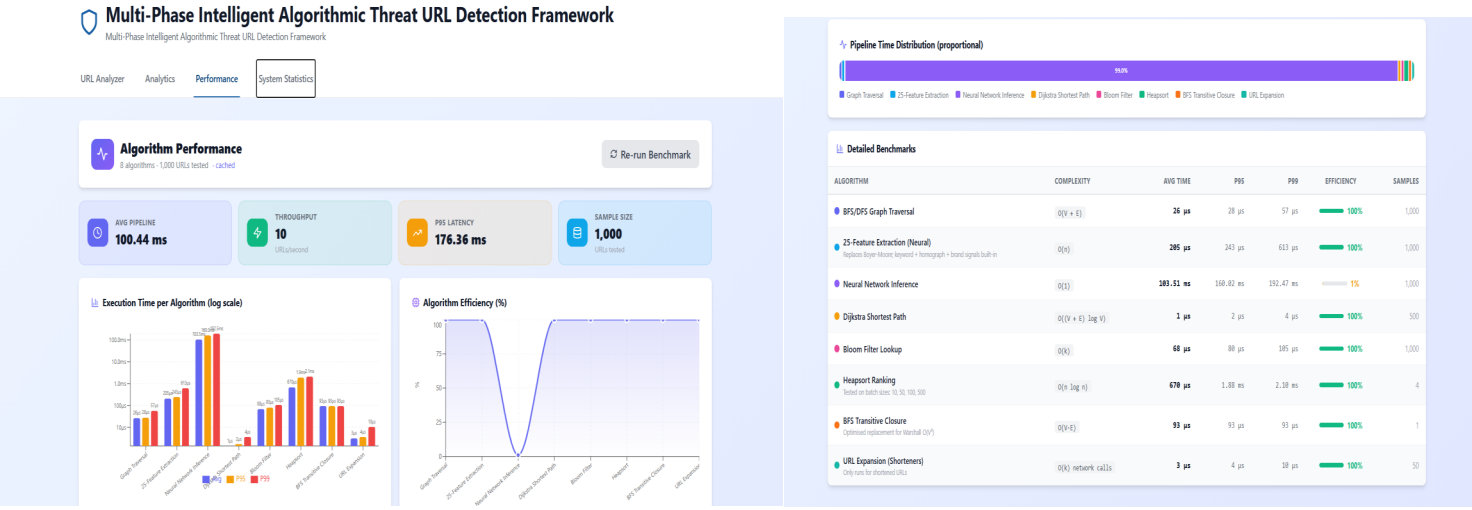

Figure 3.4: Performance Dashboard

Description: The Performance Dashboard displays benchmark results including execution time, latency, throughput, and resource utilization of different algorithms implemented in the framework.

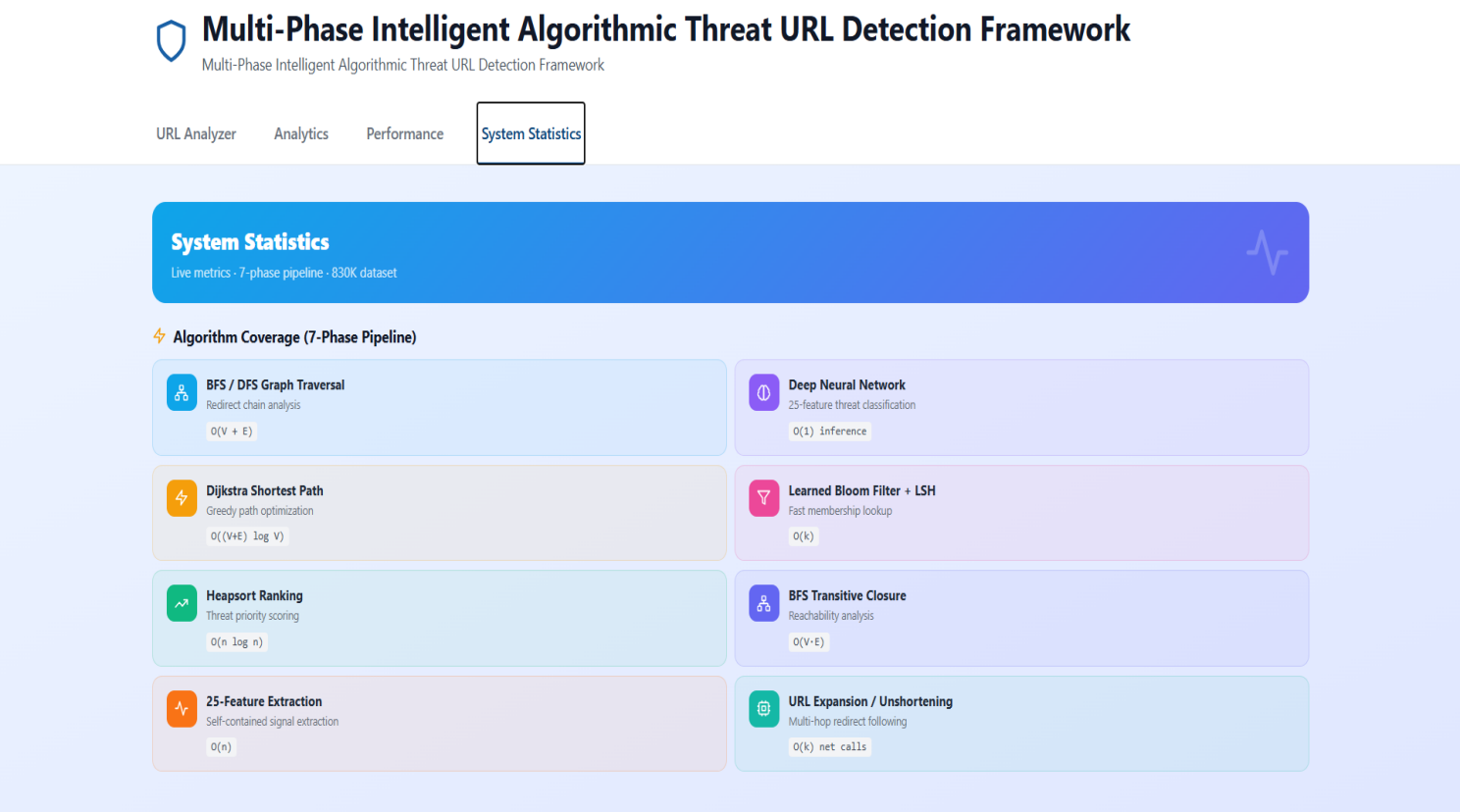

Figure 3.5: System Statistics Dashboard

Description: This dashboard summarizes all algorithms used within the framework and provides information regarding their role and computational complexity.

3.8 Coding Best Practices

Several software engineering best practices were followed during implementation. The framework follows a modular architecture in which each phase is implemented independently. This improves code organization and simplifies debugging and maintenance.

Meaningful variable names, function-based programming, proper documentation, and consistent coding standards were used to improve readability. The separation of frontend and backend components enhances scalability and allows future modifications without affecting the entire system.

The implementation also promotes reusability, enabling individual modules to be integrated into other cybersecurity applications if required.

3.9 Chapter Summary

This chapter described the implementation details of the proposed Multi-Phase Intelligent Algorithmic Threat URL Detection Framework. The framework was developed using Python, Flask, TensorFlow, React.js, and supporting technologies. A modular architecture was adopted to improve maintainability and scalability. The integration of machine learning, graph traversal algorithms, optimization techniques, blacklist verification, and ranking mechanisms resulted in a comprehensive URL threat detection system capable of providing real-time analysis and threat assessment.

CHAPTER 4
PROTOTYPE DEVELOPMENT

4.1 Approach

The proposed Multi-Phase Intelligent Algorithmic Threat URL Detection Framework was developed using a layered and modular approach that integrates Design and Analysis of Algorithms (DAA) concepts with Machine Learning techniques. The primary objective of the framework is to identify malicious URLs by performing multiple stages of analysis instead of relying on a single detection mechanism. This layered architecture improves the accuracy, reliability, and efficiency of the overall system.

The framework follows a sequential processing pipeline in which each URL passes through several analysis modules before the final threat assessment is generated. Every module performs a specific task and contributes valuable information to the overall decision-making process. The analysis begins with URL Expansion, followed by Graph Traversal, Pattern Matching, Neural Network Classification, Greedy Optimization, Bloom Filter Verification, and finally Heapsort Ranking.

The proposed system combines traditional algorithmic techniques with modern Artificial Intelligence. Graph algorithms such as Breadth First Search (BFS) and Depth First Search (DFS) analyze redirect chains and URL relationships. Boyer-Moore Pattern Matching efficiently detects phishing-related keywords embedded in URLs. A Neural Network model serves as the intelligent classifier by extracting multiple URL features and predicting the probability that the URL is malicious. Greedy Optimization prioritizes high-risk URLs, Bloom Filter performs fast blacklist verification, and Heapsort ranks URLs according to their threat scores.

The layered approach ensures that no single algorithm is solely responsible for the final decision. Instead, each module contributes partial information, which collectively improves the reliability and robustness of the threat detection process.

4.2 Prototype Development Process

The development of the prototype was carried out systematically through multiple stages.

Stage 1: Requirement Analysis

Initially, the problem of malicious URL detection was studied in detail. Existing phishing detection techniques, machine learning approaches, and algorithmic solutions were analyzed to identify their advantages and limitations. Based on this study, the system requirements and objectives were finalized.

Stage 2: System Design

The overall system architecture was designed by dividing the application into independent functional modules. A

pipeline-based architecture was selected to improve modularity, maintainability, and scalability. The responsibilities of each module were clearly defined to ensure smooth interaction among different components.

Stage 3: Backend Development

The backend was implemented using Python and Flask. Independent modules were developed for URL Expansion, Graph Traversal, Pattern Matching, Neural Network Classification, Greedy Optimization, Bloom Filter Verification, and Heapsort Ranking. REST APIs were created to facilitate communication between the backend and frontend.

Stage 4: Frontend Development

A user-friendly web interface was developed using React.js, JavaScript, Tailwind CSS, and Vite. The frontend provides multiple dashboards that allow users to analyze URLs, visualize threat scores, view benchmark results, and understand system statistics through an interactive interface.

Stage 5: Machine Learning Integration

A Neural Network model was integrated into the framework using TensorFlow and Keras. The model processes extracted URL features and predicts the likelihood that a URL is malicious. This component acts as the primary decision-making module of the framework.

Stage 6: Integration

All backend modules were integrated with the frontend using REST APIs. Proper communication between different modules was tested to ensure seamless data transfer and accurate execution of the complete analysis pipeline.

Stage 7: Testing and Validation

The developed prototype was tested using various categories of URLs, including legitimate websites, suspicious URLs, and phishing-like URLs. The generated threat scores, pipeline outputs, and dashboard visualizations were verified to ensure proper functionality. Performance benchmarking was also conducted to evaluate the efficiency of the implemented algorithms.

4.3 Working Procedure

The complete workflow of the proposed system is illustrated below.

1. The user enters a URL through the web-based interface.
2. The URL Expansion module checks whether the URL is shortened and retrieves the actual destination URL if necessary.
3. The Graph Traversal module applies BFS and DFS algorithms to analyze redirect chains and redirect depth.
4. The Pattern Matching module searches for phishing-related keywords using the Boyer-Moore algorithm.
5. The Feature Extraction process extracts multiple URL characteristics such as URL length, entropy, HTTPS status, keyword score, and suspicious domain indicators.
6. The Neural Network processes the extracted features and predicts the probability that the URL is malicious.

7. The Greedy Optimization module prioritizes URLs based on calculated threat scores.
8. The Bloom Filter checks whether the URL already exists in the blacklist of known malicious URLs.
9. The Heapsort module ranks URLs according to their threat scores.
10. Finally, all results are combined and displayed on the dashboard, where the URL is classified as Safe, Suspicious, or Malicious along with the calculated threat score.

This systematic workflow enables the framework to analyze URLs efficiently while providing detailed information about each stage of processing.

4.4 Prototype Features

The developed prototype provides the following features:

- Interactive web-based interface for URL analysis.
- Real-time threat score generation.
- URL Expansion for shortened URLs.
- Redirect chain analysis using BFS and DFS.
- Keyword detection using Boyer-Moore Pattern Matching.
- AI-based threat prediction using Neural Networks.
- Fast blacklist verification using Bloom Filter.
- Threat ranking using Heapsort.
- Performance benchmarking of implemented algorithms.
- Analytics and system statistics dashboards.

These features make the prototype suitable for demonstrating practical applications of DAA concepts in cybersecurity.

4.5 Chapter Summary

This chapter described the methodology followed for developing the proposed framework. The prototype was implemented using a modular architecture consisting of multiple algorithmic phases integrated with a machine learning model. The complete development process, implementation strategy, system workflow, and prototype features were discussed. The successful integration of DAA algorithms with Artificial Intelligence demonstrates the effectiveness of the proposed solution for malicious URL detection.

CHAPTER 5
EXPECTED OUTCOMES

5.1 Functional Prototype Development

The primary outcome of this project is the successful development of a fully functional web-based prototype capable of analyzing URLs and detecting potential phishing attacks. The developed system allows users to enter a URL through an interactive interface and receive an immediate threat assessment. The generated output includes the threat score, classification result, pipeline analysis, feature analysis, and performance statistics.

The prototype demonstrates how multiple algorithms can work together within a single framework to solve a practical cybersecurity problem.

5.2 Improvement in Performance and Efficiency

The proposed framework improves the efficiency of malicious URL detection by combining multiple algorithms that perform different tasks throughout the analysis pipeline.

Graph Traversal algorithms efficiently analyze redirect chains, while Boyer-Moore Pattern Matching performs fast keyword detection. The Neural Network provides intelligent classification based on extracted URL features. Bloom Filter enables rapid blacklist verification with minimal memory consumption, and Heapsort provides efficient threat ranking for multiple URLs.

Performance benchmarking demonstrates the execution time of each algorithm, allowing developers to identify computational bottlenecks and optimize the system further. Overall, the proposed framework achieves efficient processing while maintaining accurate threat detection.

5.3 Technical Innovation

One of the major contributions of this project is the integration of classical Design and Analysis of Algorithms with modern Machine Learning techniques. Unlike conventional URL detection systems that rely only on blacklist verification, the proposed framework performs multi-stage analysis before generating a decision.

The project combines Graph Algorithms, Pattern Matching, Optimization Techniques, Probabilistic Data Structures, Sorting Algorithms, and Neural Networks within a single cybersecurity application. This integration demonstrates how algorithmic concepts taught in DAA can be applied to solve real-world security problems.

5.4 Societal and Industrial Impact

The developed framework has significant practical applications in cybersecurity and internet safety. It can help protect users against phishing attacks, malicious websites, and online fraud by identifying suspicious URLs before

users access them.

The proposed solution can be integrated into web browsers, corporate security gateways, educational institutions, email filtering systems, and enterprise cybersecurity platforms. It can also improve awareness regarding phishing attacks and promote safer internet usage.

From an industrial perspective, the framework can assist organizations in reducing cybersecurity risks, minimizing financial losses, and improving overall network security.

5.5 Contribution to Research and Future Scope

The project demonstrates the practical implementation of several important DAA concepts within a real-world cybersecurity application. It provides a foundation for future research in intelligent threat detection and machine learning-based cybersecurity systems.

Future improvements may include:

- Integration of larger phishing datasets for improved model accuracy.
- Automatic updating of malicious URL databases.
- Real-time threat intelligence integration.
- Browser extension development for live URL scanning.
- Mobile application development.
- Cloud deployment for large-scale URL analysis.
- Advanced Deep Learning models such as LSTM or Transformer-based architectures.
- Integration with enterprise cybersecurity platforms and Security Information and Event Management (SIEM) systems.

These enhancements will improve scalability, detection accuracy, and practical usability of the framework.

5.6 Conclusion

The **Multi-Phase Intelligent Algorithmic Threat URL Detection Framework** successfully demonstrates the application of Design and Analysis of Algorithms in the field of cybersecurity. The framework integrates BFS, DFS, Boyer-Moore Pattern Matching, Greedy Optimization, Bloom Filter, Heapsort, and Neural Network Classification to provide an intelligent and efficient URL threat detection system.

The developed prototype is capable of analyzing URLs through multiple stages, generating threat scores, and classifying URLs as Safe, Suspicious, or Malicious.

The project highlights the importance of combining classical algorithms with Artificial Intelligence to address modern cybersecurity challenges.The proposed framework serves as an effective educational model for demonstrating DAA concepts while also providing a practical solution that can be extended for future research and real-world cybersecurity applications.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., Cambridge, MA, USA: MIT Press, 2022.

[2] M. Sahingoz, E. Buber, O. Demir and B. Diri, "Machine Learning Based Phishing Detection from URLs," *Expert Systems with Applications*, vol. 117, pp. 345-357, 2019.

[3] J. Ma, L. K. Saul, S. Savage and G. M. Voelker, "Beyond Blacklists: Learning to Detect Malicious Web Sites from Suspicious URLs," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2009, pp. 1245-1254.

[4] B. H. Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422-426, 1970.

[5] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269-271, 1959.

[6] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, Cambridge, MA, USA: MIT Press, 2016.

[7] F. Chollet, *Deep Learning with Python*, 2nd ed., Shelter Island, NY, USA: Manning Publications, 2